12

关于内存虚拟化的对话

学生：那么，虚拟化（*virtualization*）讲完了吗？

教授：没有！

学生：嘿，没必要这么激动吧；我只是问个问题而已。学生本来就该提问的，对吧？

教授：嗯，教授们确实总这么说，但他们真正的意思是：**如果**是好问题，**而且**你确实动脑子想过，那就问吧。

学生：唉，这话可真让我泄气。

教授：目的达到了。不管怎样，虚拟化还远远没讲完！你刚刚看到的只是怎样虚拟化*CPU*，真正的大怪物还躲在衣柜里：内存。虚拟化内存很复杂，需要我们弄懂硬件和操作系统之间更多错综复杂的交互细节。

学生：听起来很酷。为什么这么难？

教授：嗯，细节很多，你得在脑子里把它们理清楚，才能真正建立起关于整个过程的心智模型。我们会从简单的开始，先讲像基址（*base*）/界限（*bounds*）这样非常基本的技术，再慢慢增加复杂度，应对新的挑战，其中包括转换后备缓冲区（*TLB*）和多级页表（*multi-level page table*）这些有趣的话题。最后，我们就能讲清楚一个功能完整的现代虚拟内存（*virtual memory*）管理器是怎样工作的。

学生：太好了！对于被这么多信息淹没、又总是睡眠不足的可怜学生，您有什么建议吗？

教授：睡眠不足好办：多睡觉（少开派对）。至于理解虚拟内存，先从这一点开始：**用户程序产生的每一个地址都是虚拟地址**。操作系统只是给每个进程（*process*）营造了一个假象，具体来说，就是让它以为自己拥有一大块私有的内存；在一些硬件的帮助下，操作系统会把这些假装的虚拟地址（*virtual address*）转换成真实的物理地址（*physical address*），从而找到所需的信息。

学生：好，我想我能记住……（自言自语）用户程序的每个地址都是虚拟的，用户程序的每个地址都是虚拟的，每个……

教授：你在嘀咕什么呢？

学生：哦，没什么……（尴尬的停顿）……话说回来，操作系统为什么又要提供这种假象来着？

教授：主要是为了**易用性**（*ease of use*）：操作系统会让每个程序都觉得自己有一个很大的连续**地址空间**（*address space*），可以把代码和数据放进去；因此，作为程序员，你永远不用操心诸如“这个变量该放在哪儿?”之类的事情，因为程序的虚拟地址空间（*virtual address space*）很大，有的是地方放这些东西。如果你得操心怎样把所有代码和数据塞进一块又小又挤的内存里，程序员的日子就要难过得多。

学生：还有别的原因吗？

教授：嗯，**隔离**（*isolation*）和**保护**（*protection*）也是大事。我们可不希望一个出错的程序能读取，或者更糟，能覆盖别的程序的内存，对吧？

学生：大概不希望吧。除非那个程序是你讨厌的人写的。

教授：嗯……我觉得下学期你的课表里可能得加一门道德伦理课了。也许操作系统课没有把正确的道理传达给你。

学生：也许应该加。不过别忘了，教我们“对付出错的进程，操作系统的正确做法就是杀掉这个违规进程”的，可不是我！

 © 2008–23, ARPACI-DUSSEAU